# 7 Funkce

**V této kapitole se dozvíte:**

- Jak vytvořit uživatelsky definovanou funkci v PHP.
- Jakým způsobem se předávají parametry v PHP.
- Jaké jsou nejdůležitější předdefinované funkce v PHP.

**Po jejím prostudování byste měli být schopni:**

- Vytvořit vlastní uživatelsky definovanou funkci.
- Umět použít ve funkci předávání parametrů hodnotou.
- Umět použít ve funkci předávání parametrů odkazem.

**Klíčová slova této kapitoly:**

funkce, function, return, předávání parametrů.

**Doba potřebná ke studiu: 8 hodiny**

## Průvodce studiem




*Studium této kapitoly předpokládá základní znalosti algoritmizace, konkrétně tvorbu uživatelských funkcí (metod, procedur). Při volání funkce mohou být parametry předány dvěma způsoby – hodnotou nebo odkazem. Ve druhé části se seznámíte s nejdůležitějšími předdefinovanými funkcemi jazyka PHP, které Vám mohou usnadnit tvorbu Vašich aplikací.*

Funkce v jazyce PHP představují určitý blok kódu, který je pouze jednou nadefinován a může být pak v jiných částech programu vyvolán. V jiných programovacích jazycích bývají někdy funkce také nazývány jako metody. Funkce většinou mají jako vstup argument nebo argumenty, které bývají označovány také jako parametry. Funkce provádí předdefinovanou sadu operací a vrací výslednou hodnotu. Pomocí funkcí můžeme vytvářet modulární aplikace. Kód, který by se v aplikaci opakoval na více místech, je nadefinován jako funkce a na potřebných místech je pouze zavolán.

Používání funkcí šetří čas a snižuje chybovost kódu. Jazyk PHP obsahuje velké množství předdefinovaných funkcí, s některými z nich jsme se již setkali. Mezi předdefinované funkce jazyka PHP patří například `gettype()`, `count()`, `isset()` nebo `defined()`. S dalšími předdefinovanými funkcemi se seznámíte na konci této kapitoly. Kromě předdefinovaných funkcí je možné v PHP vytvářet uživatelsky definované funkce. S uživatelsky definovanými funkcemi se seznámíte v následujících odstavcích.

## 7.1 Deklarace, definice a volání funkce

Deklarace funkce se provádí příkazem `function`, za kterým následuje název funkce (identifikátor funkce). Pro názvy funkcí platí stejná pravidla jako pro

názvy proměnných, tj. je možné používat písmena anglické abecedy, číslice a znak podtržítka. Za názvem funkce následuje seznam parametrů (argumentů) funkce, který je uveden v kulatých závorkách. Jednotlivé parametry jsou odděleny čárkami. Za kulatými závorkami následují složené závorky, které obsahují tělo funkce. Tělo funkce tvoří kód, který má funkce provádět, a označuje se jako definice funkce. Funkce nemusí obsahovat žádné parametry, ale za jejím názvem musí být uvedeny prázdné kulaté závorky. Funkce může vracet nějakou hodnotu. Pokud má funkce vracet hodnotu, musí tělo funkce obsahovat příkaz `return`, který bývá zpravidla uveden na konci těla funkce.

Volání funkce provedeme uvedením jejího názvu, za kterým následují parametry uvedené v kulatých závorkách. Pokud funkce vrací nějakou hodnotu, měla by být tato hodnota uložena do nějaké proměnné nebo jinak zpracována. Při volání funkce je samozřejmě potřeba dodržet počet parametrů, které funkce obsahuje.

Následující výpis obsahuje obecnou deklaraci, definici a volání funkce.

**Výpis 7.1 Obecná definice a volání funkce.**



```
<?php
  // deklarace a definice funkce
  function nazev_funkce(parametry) {
    ...tělo funkce...
  }

  // volání funkce
  nazev_funkce(parametry);
?>
```

Nyní si uveďme konkrétní příklady funkce. Nejprve to bude funkce, která nevrací žádnou hodnotu. Funkce má jako parametr text, který představuje titulek stránky v HTML. Text je vložen mezi značky `<title>` a `</title>` a je vypsán do výstupu v prohlížeči. Druhá funkce má dva parametry a vrací jejich součet. Uvedení závorek u součtu v příkazu `return` není nutné a je uvedeno pouze pro přehlednost.

**Výpis 7.2 Příklad konkrétních funkcí.**

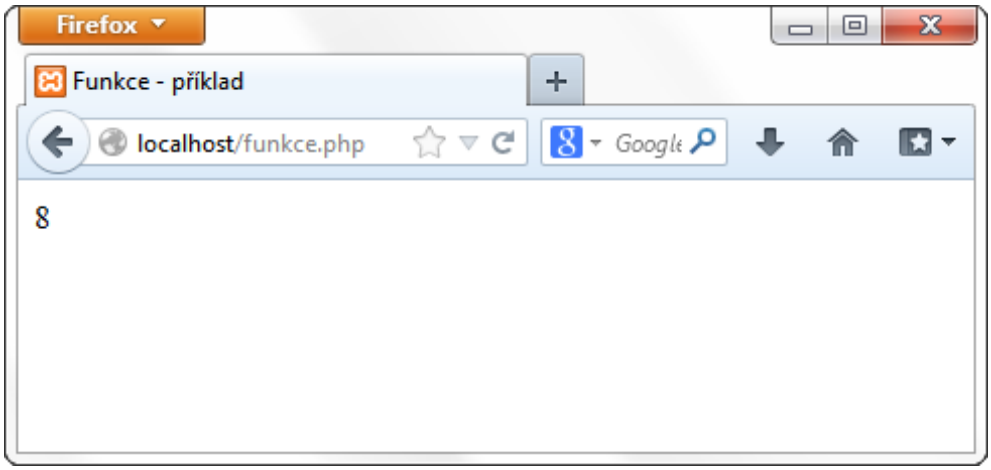


```
<?php
  function titulek($text) {
    echo "<title>" . $text . "</title>";
  }
```

```
  titulek("Funkce - příklad");

  function soucet($a, $b) {
    return ($a + $b);
  }

  echo soucet(5, 3);
?>
```




**Obrázek 7.1 Ukázka použití funkce.**

## 7.2 Předávání parametrů

Předávání parametrů (argumentů) v PHP funguje úplně stejně jako ve většině jiných programovacích jazyků. V deklaraci funkce jsou v závorce uvedeny formální parametry, jejichž identifikátory (názvy) v podstatě pouze představují symboly používané v těle funkce. Při volání funkce jsou formální parametry nahrazeny skutečnými parametry, tj. skutečnými hodnotami, které funkcí „posíláme“. Při předávání parametrů funkci rozlišujeme předávání parametrů hodnotou a předávání odkazem.



Jazyk PHP používá implicitně předávání parametrů hodnotou. Při volání funkce je v rámci vykonávání těla funkce vytvořena lokální kopie parametru, se kterou se pracuje. Pokud tuto hodnotu uvnitř funkce změníme, nemá to na předávanou hodnotu žádný vliv. Následující kód představuje ukázku použití předávání parametru hodnotou a jeho efekt. Obrázek pak zobrazuje výsledek v prohlížeči.

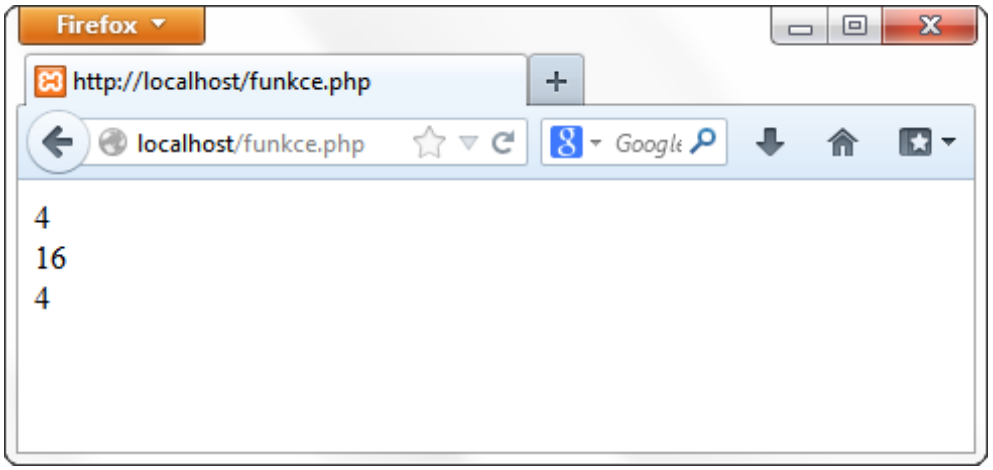
**Výpis 7.3 Předávání parametrů hodnotou.**

```
<?php
  function mocnina($x) {
    $x = $x * $x;
```



```
    echo $x . "<br>";
  }

  $a = 4;
  echo $a . "<br>";
  mocnina($a);
  echo $a . "<br>";
?>
```




**Obrázek 7.2 Předávání parametrů hodnotou.**

Vidíme, že nejprve je vypsána původní hodnota proměnné `$a`. Následně je volána funkce `mocnina()`, ve které je předaný parametr umocněn a vytištěn na obrazovku. Hodnota parametru je `4`, takže se vytiskne číslo `16`. Nakonec je opět vytištěna hodnota proměnné `$a`. Je vytištěna původní hodnota, protože umocnění se provedlo s lokální kopií předaného parametru. Tato kopie zanikla při ukončení provádění těla funkce.



Druhou možností, jak předávat v PHP parametry funkcím, je předávání odkazem. Funkci není předávána hodnota parametru, ale odkaz na původní předávanou hodnotu. Efekt tohoto způsobu předávání parametrů je, že změny parametru uvnitř těla funkce se projeví i mimo funkci. Pokud chceme parametr předat odkazem, je potřeba před jeho identifikátor umístit znak `&` (ampersand). Následující výpis ukazuje použití parametru předaného odkazem, na obrázku je pak zobrazen výsledek zobrazený v prohlížeči.

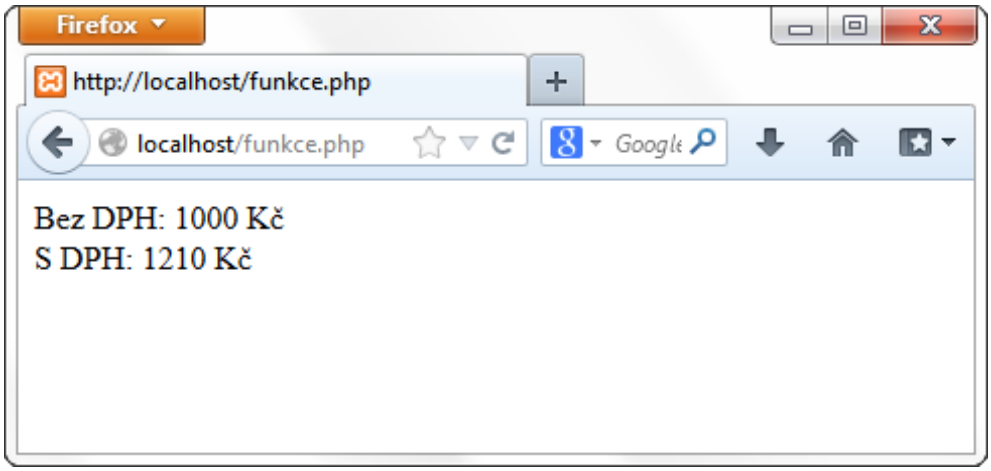
**Výpis 7.4 Předávání parametru odkazem.**



```
<?php
  function pricti_dph(&$x) {
    $x = $x + $x * 0.21;
  }
```

```
  $cena = 1000;
  echo "Bez DPH: " . $cena . " Kč<br>";
  pricti_dph($cena);
  echo "S DPH: " . $cena . " Kč<br>";
?>
```

Na následujícím obrázku je zobrazen výstup v prohlížeči. Vidíme, že hodnota proměnné `$cena` se po zavolání funkce `pricti_dph()` změnila a druhý výpis již vypíše původní hodnotu zvýšenou o DPH, tj. o 21 %.

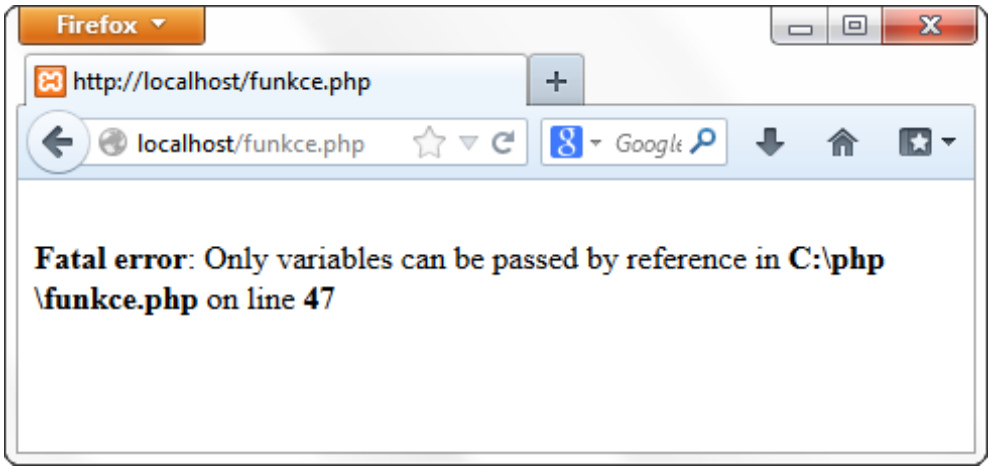



**Obrázek 7.3 Předávání parametrů odkazem.**

Při předávání parametrů hodnotou mohou být předány i konstanty nebo literály. Ve výpisu 7.2 je při volání funkce `soucet()` použito předání konstant, v tomto případě se jedná o hodnoty `5` a `3`. Při předávání parametrů odkazem ale musí být předána hodnota proměnné, předání konstanty způsobí chybu. Pokud bychom funkci `pricti_dph()`, která byla definována ve výpisu 7.4, zavolali následujícím způsobem, výsledkem bude chybová zpráva zobrazená na obrázku.

**Výpis 7.5 Volání funkce, které způsobí chybu.**

```
<?php
  pricti_dph(100);
?>
```




**Obrázek 7.4 Chybová zpráva.**

Jazyk PHP umožňuje u některých parametrů stanovit výchozí hodnoty. Přiřazení hodnoty se provádí v deklaraci funkce. Parametr, který má nastavenu výchozí hodnotu, je nepovinný při volání funkce. Pokud funkce obsahuje povinné i nepovinné parametry, je potřeba v deklaraci funkce nejprve uvést povinné parametry a teprve poté parametry nepovinné. Následující výpis ukazuje modifikaci funkce `pricti_dph()`, která obsahuje parametr s nastavenou výchozí hodnotou, tj. nepovinný parametr.

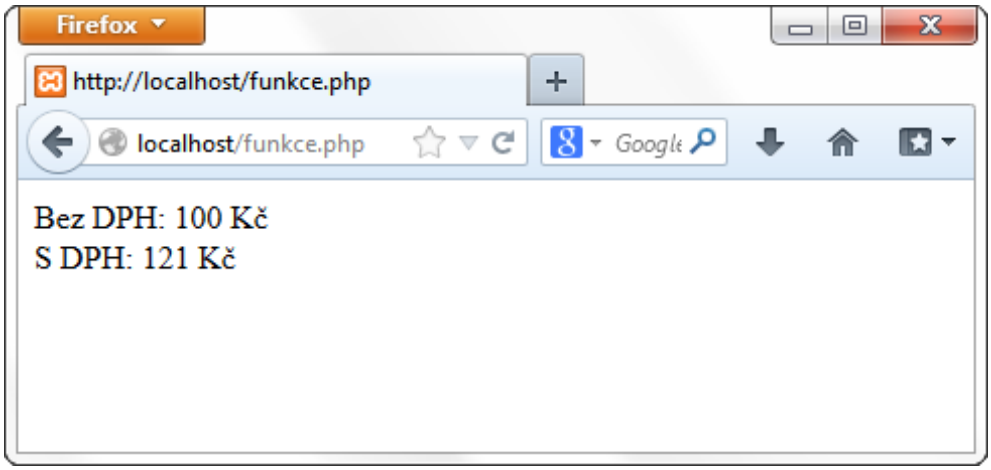
**Výpis 7.6 Funkce s nepovinným parametrem.**



```
<?php
  function pricti_dph(&$x, $sazba = 0.21) {
    $x = $x + $x * $sazba;
  }

  $cena = 100;
  echo "Bez DPH: " . $cena . " Kč<br>";
  pricti_dph($cena);
  echo "S DPH: " . $cena . " Kč<br>";
?>
```

Jak vidíme, druhý parametr funkce `pricti_dph()` má nastavenou výchozí hodnotu, touto hodnotou je `0.21`. Při volání funkce tedy stačí uvést pouze jeden parametr, v tomto případě je parametrem hodnota proměnné `$cena`. Výsledek uvedeného kódu zobrazený v prohlížeči zobrazuje následující obrázek.




**Obrázek 7.5 Funkce s nepovinným parametrem.**

## 7.3 Rozsah platnosti proměnných



S problematikou tvorby uživatelsky definovaných funkcí souvisí i rozsah platnosti proměnných, který určuje, ve kterých částech programu je proměnná přístupná. Proměnné vytvořené uvnitř těla funkce, jsou lokální pouze v rámci této funkce. Pokud ve funkce vytvoříme proměnnou se stejným identifikátorem, jako má některá proměnná mimo funkci, jedná se o novou proměnnou.

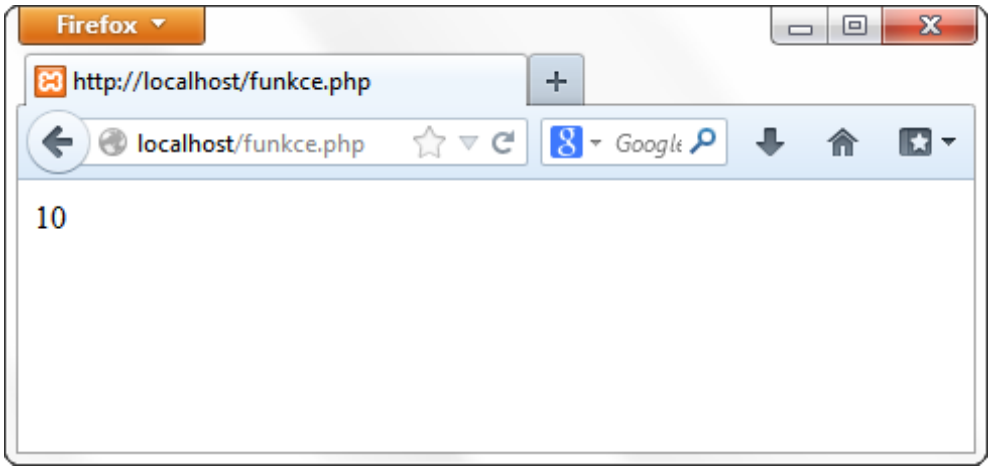
**Výpis 7.7 Ukázka lokální proměnné v rámci funkce.**



```
<?php
  function zmen_hodnotu() {
    $cislo = 20;
  }

  $cislo = 10;
  zmen_hodnotu();
  echo $cislo;
?>
```

Funkce `zmen_hodnotu()` přiřazuje do proměnné `$cislo` hodnotu `20`. Mimo funkci vytvoříme globální proměnnou, která se jmenuje také `$cislo` a přiřadíme jí do ní hodnotu `10`. Po zavolání funkce `zmen_hodnotu()` se ale hodnota proměnné `$cislo` nezmění, protože při volání funkce je vytvořená nová lokální proměnná. I když se obě proměnné jmenují stejně, jedná se o dvě různé proměnné. Výsledkem vykonání uvedeného kódu tedy bude vypsání čísla `10`, jak ukazuje následující obrázek.




**Obrázek 7.6 Ukázka lokální proměnné.**

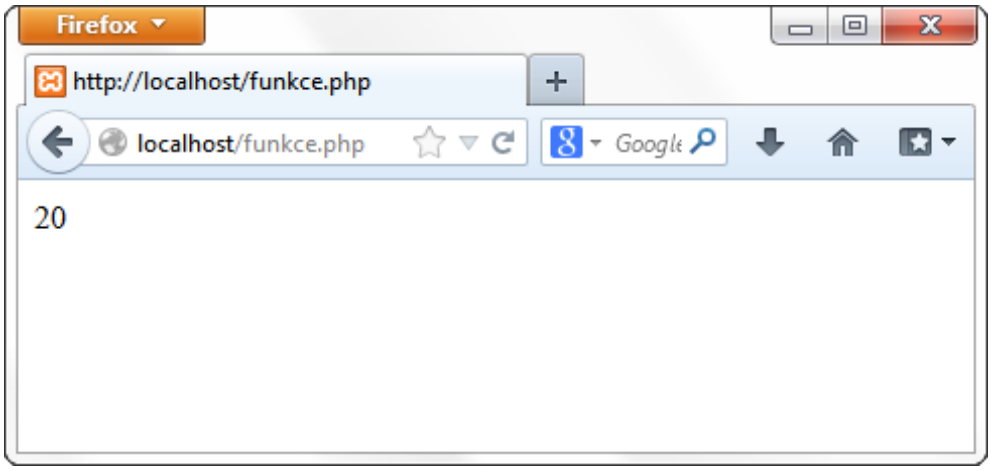
Pokud chceme v těle funkci přistupovat ke globální proměnné definované mimo funkci, je potřeba použít příkaz `global`. Tímto příkazem říkáme, že v rámci funkce nechceme vytvořit novou proměnnou, ale že chceme použít proměnnou, která je dostupná mimo tělo funkce. Následující výpis je velmi podobný výpisu 7.7, pouze je v těle funkce `zmen_hodnotu()` použitý příkaz `global`. Výsledkem provedení tohoto kódu je vypsání čísla `20`, jak opět ukazuje následující obrázek.

**Výpis 7.8 Použití příkazu global.**



```
<?php
  function zmen_hodnotu() {
    global $cislo;
    $cislo = 20;
  }

  $cislo = 10;
  zmen_hodnotu();
  echo $cislo;
?>
```




**Obrázek 7.7 Použití příkazu global.**

Alternativou použití příkazu `global` je použití superglobálního pole `$GLOBALS`, které obsahuje všechny globální proměnné programu (skriptu). V následujícím výpisu je uvedena ukázka použití tohoto pole ve funkci `zmen_hodnotu()`. U názvu proměnné v hranatých závorkách již není nutné používat znak `$` před názvem proměnné. V případě, že zbylý kód bude stejný jako v předchozím výpisu, vypíše se v prohlížeči číslo `20`.

**Výpis 7.9 Ukázka použití superglobálního pole.**

```
<?php
  function zmen_hodnotu() {
    $GLOBALS["cislo"] = 20;
  }
?>
```

Jazyk PHP umožňuje definovat v rámci funkcí statické proměnné, které si uchovávají svou předchozí hodnotu mezi jednotlivými voláními funkce. Statickou proměnnou vytvoříme pomocí klíčového slova `static`, které uvedeme v deklaraci proměnné.

## 7.4 Vnořené funkce a rekurze

Jazyk PHP umožňuje používat vnořené funkce, tzn. že v rámci jedné funkce vytvoříme další funkci. Všechny funkce, včetně funkcí vnořených do jiných funkcí, jsou v PHP globální, tzn. je možné je volat i mimo funkci, ve které jsou definovány. Používáním vnořených funkcí se ovšem snižuje přehlednost kódu a je pak složitější se v něm vyznat.

Pokud funkce volá sebe sama, označujeme ji jako rekurzivní funkci. Řešení problému je pak rozděleno na menší úlohy, které jsou poté několikrát

opakovány. U rekurze je potřeba dávat pozor, aby volání funkce skončilo a nedošlo k „zacyklení“, tzn. že provádění funkce se nikdy nezastaví. Příkladem problému, kdy je možné použít rekurzi, je výpočet n-té mocniny čísla nebo výpočet faktoriálu.

## 7.5 Předdefinované funkce



Jak již bylo uvedeno, jazyk PHP obsahuje velké množství předdefinovaných funkcí. V této studijní opoře jste se již seznámili s některými z nich. Nyní si uvedeme nejdůležitější skupiny funkcí, které jsou v PHP k dispozici. Kompletní seznam funkci je možné nalézt v dokumentaci k jazyku PHP.

Pro snadnější tvorbu aplikací v jazyce PHP jsou k dispozici předdefinované funkce, které je možné rozdělit do skupin podle toho, k čemu slouží. Mezi nejdůležitější skupiny operací při tvorbě webových aplikací patří práce s polem, s řetězci, se soubory, s kalendářním datem a časem, s databází, s obrázky, matematické operace a další.

Pro práci s polem jsou k dispozici například funkce pro procházení polí, vyhledávání hodnot v poli, řazení polí, získání indexů pole (pole v PHP může být asociativní, tj. indexy pole netvoří celá čísla, ale řetězce). Funkce pro práci s textovými řetězci umožňují například zjistit délku řetězce, rozdělit řetězec, najít v řetězci znak nebo znaky, nahradit v řetězci znak nebo znaky. Funkce pro práci se soubory umožňují vytvořit soubor (na serveru), otevřít soubor, číst ze souboru a zapisovat do něj. V případě kalendářního data můžeme zjistit aktuální datum a čas a dále pak zjistit jednotlivé části zadaného data a času – můžeme například zjistit, jaký je měsíc, den, rok apod. V případě práce s aktuálním datem a časem je potřeba myslet na to, že se jedná o datum a čas na serveru. Pokud je server umístěn v jiném časovém pásmu, aktuální datum a čas se samozřejmě liší od data a času v počítači. Pro práci s databází jsou k dispozici funkce pro připojení k databázi, provedení dotazu nad databází, procházení výsledků dotazu, vložení záznamu do tabulky apod. V jazyce PHP je možné vytvářet obrázky a vkládat je do webových stránek. Mezi základní operace patří vytvoření nebo smazání obrázku, změna velikosti obrázku, kreslení základních geometrických tvarů, práce s písmy nebo práce s barvami. V oblasti matematických funkcí jsou k dispozici například goniometrické funkce, absolutní hodnota, logaritmus, mocnina, odmocnina apod.

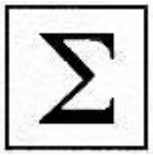
S některými předdefinovanými funkcemi se seznámíte dále v této studijní opoře.

**Kontrolní otázky:**



1. K čemu slouží funkce?
2. Jak vytvoříte funkci v jazyce PHP?
3. Jakým způsobem je možné předávat parametry funkcím v PHP?
4. Co je to vnořená funkce?
5. Co je to rekurzivní funkce?

**Shrnutí obsahu kapitoly**



V této kapitole jste s předdefinovanými funkcemi v PHP i s uživatelsky definovanými funkcemi. Uživatelsky definované funkce umožňují vytvářet modulární aplikace a snižují chybovost kódu. Definice funkce začíná klíčovým slovem `function`. Předávání parametrů v jazyce PHP je možné hodnotou a nebo odkazem. Jazyk PHP umožňuje také definici vnořených nebo rekurzivních funkcí. Pro usnadnění tvorby aplikací je v jazyce PHP k dispozici velké množství předdefinovaných funkcí. Nejdůležitějšími skupinami jsou funkce pro práci s polem, se soubory, s řetězci, s databází nebo matematické funkce.